EECS 348 Assignment 5 — GenAI Analysis

Student: Justin Whitaker | **Date:** October 29, 2026 | **Program:** Assignment5.cpp, my revision of Aidan Wiehl's Assignment 4 recursive, object-oriented Sudoku solver
Task given to the GenAIs: modify the received Assignment 4 code so that each puzzleN.txt also produces solutionN.txt containing every solution and "Solutions found: n", or "No solution found."
Report attribution: I wrote this report with help from Claude Code (Claude Opus 5.5). Every number below comes from programs compiled and run on my computer.

a. & b. The Two GenAI Programs and How I Accessed Them

GenAI program	Company	How I accessed it
Gemini 4 Argon (Medium reasoning setting)	Google	Google Antigravity command-line tool (`agy.exe`) installed on my Windows 11 computer, in non-interactive print mode: `agy --model gemini-4-argon-medium -p "<prompt>"`. I ran it from an empty folder so it could not see any other files, and saved its full reply exactly as returned.
Claude Opus 5.5 (Medium reasoning setting)	Anthropic	The same Antigravity command-line tool on my computer: `agy --model claude-opus-5-5-medium -p "<prompt>"`, also from its own empty folder, with the full reply saved exactly as returned.

Both programs were run on October 29, 2026 with the prompt text read from the same file, so the model was the only difference between the two runs. They come from different companies (I did not pair ChatGPT with Copilot).

c. Prompt Used (identical for both GenAIs)

The full prompt is shown below exactly as sent. It includes the complete Assignment 4 source code I received.

```
I was given another student's EECS 348 Assignment 4 program (a recursive, object-oriented C++ Sudoku solver). For
Assignment 5 I must modify that program, not replace it. Please modify the code below so that it meets these
requirements:

1. Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle number (puzzle1.txt -
> solution1.txt, ..., puzzle5.txt -> solution5.txt).
2. Each solutionN.txt file must contain:
   - If no solutions are found, a single line: "No solution found."
   - If solutions are found, all solutions followed by a single line: "Solutions found: n", where n is the number of
solutions found.
   - The solutions must be in the same format as the input files puzzle1.txt - puzzle5.txt (an example input file is
shown below).
3. All required Assignment 4 functionality must still work: for each puzzle the program prints the puzzle file name,
the original puzzle, and every solution (or "No solution found" if there is none). The solver must stay recursive
(depth-first search with backtracking) and object-oriented. If the code below has bugs that break these requirements,
fix them.
4. Modify the existing code instead of rewriting the whole file (my grader uses GitHub Blame to see what I changed),
so keep the existing classes and lines wherever possible.
5. Add a prologue comment with a "Revised by:" line and a revision date, and comment the code you add.
6. The program must compile with g++ and run on Linux (the EECS cycle servers).

Example input file (puzzle1.txt); an underscore marks an empty cell:
5 _ _ _ _ 1 7 _
1 _ 6 5 _ 9 _ 4 _
4 7 2 1 _ 6 _ _ _
9 _ _ _ _ _ 5 _ _
_ 1 8 _ 9 5 4 _ _
6 _ _ 4 _ 2 3 8 9
_ 4 _ _ _ _ 9 3 _
_ 9 _ 7 _ 3 _ 5 _
2 6 3 9 5 8 7 1 4
```

Assignment 4 code to modify:
```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

class Sudoku {
private:
    vector<vector<char>> grid;

public:
    Sudoku() {
        grid.resize(9, vector<char>(9));
    }

    bool loadPuzzle(const string& filename) {
        ifstream fin(filename);

        if (!fin) {
            cout << "Error opening file: " << filename << endl;
            return false;
        }

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                fin >> grid[row][col];
            }
        }

        fin.close();
        return true;
    }

    void printBoard() const {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                cout << grid[row][col] << " ";
            }
            cout << endl;
        }
    }

    bool findEmptyCell(int& row, int& col) const {

        for (row = 0; row < 9; row++) {
            for (col = 0; col < 9; col++) {

                if (grid[row][col] == '_')
                    return true;
            }
        }

        return false;
    }

    bool validInRow(int row, char value) const {

        for (int col = 0; col < 9; col++) {
            if (grid[row][col] == value)
                return false;
        }

        return true;
    }

    bool validInColumn(int col, char value) const {

        for (int row = 0; row < 9; row++) {
            if (grid[row][col] == value)
```

```
                return false;
        }

        return true;
    }

    bool validInBlock(int row, int col, char value) const {

        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {

                if (grid[r][c] == value)
                    return false;
            }
        }

        return true;
    }

    bool isValidMove(int row, int col, char value) const {

        return validInRow(row, value) &&
               validInColumn(col, value) &&
               validInBlock(row, col, value);
    }

    void setValue(int row, int col, char value) {
        grid[row][col] = value;
    }

    vector<vector<char>> getGrid() const {
        return grid;
    }
};

class SudokuSolver {
private:
    vector<vector<vector<char>>> solutions;

public:

    void solve(Sudoku& puzzle) {

        int row;
        int col;

        if (!puzzle.findEmptyCell(row, col)) {

            solutions.push_back(puzzle.getGrid());
            return;
        }

        for (char digit = '1'; digit <= '9'; digit++) {

            if (puzzle.isValidMove(row, col, digit)) {

                puzzle.setValue(row, col, digit);

                solve(puzzle);

                // Backtracking step
                puzzle.setValue(row, col, '_');
            }
        }
    }

    void printSolutions() const {

        if (solutions.empty()) {
```

```
            cout << "\nNo solution found.\n";
            return;
        }

        for (size_t i = 0; i < solutions.size(); i++) {

            cout << "\nSolution " << i + 1 << ":\n\n";

            for (int row = 0; row < 9; row++) {

                for (int col = 0; col < 9; col++) {

                    cout << solutions[i][row][col] << " ";
                }

                cout << endl;
            }
        }

        cout << "\nSolutions found: "
             << solutions.size()
             << endl;
    }
};

class PuzzleManager {
private:
    vector<string> puzzleFiles;

public:

    PuzzleManager() {

        puzzleFiles.push_back("puzzle1.txt");
        puzzleFiles.push_back("puzzle2.txt");
        puzzleFiles.push_back("puzzle3.txt");
        puzzleFiles.push_back("puzzle4.txt");
        puzzleFiles.push_back("puzzle5.txt");
    }

    void processPuzzles() {

        for (string filename : puzzleFiles) {

            Sudoku puzzle;

            if (!puzzle.loadPuzzle(filename))
                continue;

            cout << "\n================================================";
            cout << "\nPuzzle File: " << filename;
            cout << "\n================================================\n";

            cout << "\nOriginal Puzzle:\n\n";
            puzzle.printBoard();

            SudokuSolver solver;
            solver.solve(puzzle);

            cout << "\nSolved Puzzle(s):\n";
            solver.printSolutions();

            cout << "\n";
        }
    }
};

int main() {

    PuzzleManager manager;

    manager.processPuzzles();
```

```
    return 0;
}
```

Reply with the complete modified program as a single C++ code block. Do not create or edit any files; just reply with the code.

d. Generated Code from Both GenAIs

GenAI 1 — Gemini 4 Argon: complete, unedited reply

```cpp
```cpp
/*
 * EECS 348 - Assignment 5: Recursive Object-Oriented Sudoku Solver
 * Description: Reads Sudoku puzzles (puzzle1.txt - puzzle5.txt), solves them
 *              recursively using depth-first search with backtracking, prints
 *              the results to standard output, and writes the solutions to
 *              corresponding output files (solution1.txt - solution5.txt).
 * Input:       puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
 * Output:      Console output of puzzles and solutions; solution1.txt - solution5.txt
 * Original Author: Assignment 4 Student
 * Revised by:  [Your Name]
 * Revision Date: October 29, 2026
 */

#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

class Sudoku {
private:
    vector<vector<char>> grid;

public:
    Sudoku() {
        grid.resize(9, vector<char>(9));
    }

    bool loadPuzzle(const string& filename) {
        ifstream fin(filename);

        if (!fin) {
            cout << "Error opening file: " << filename << endl;
            return false;
        }

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                fin >> grid[row][col];
            }
        }

        fin.close();
        return true;
    }

    void printBoard() const {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                cout << grid[row][col] << " ";
            }
            cout << endl;
        }
    }

    bool findEmptyCell(int& row, int& col) const {

        for (row = 0; row < 9; row++) {
            for (col = 0; col < 9; col++) {

                if (grid[row][col] == '_')
                    return true;
            }
```
```

```
        }

        return false;
    }

    bool validInRow(int row, char value) const {

        for (int col = 0; col < 9; col++) {
            if (grid[row][col] == value)
                return false;
        }

        return true;
    }

    bool validInColumn(int col, char value) const {

        for (int row = 0; row < 9; row++) {
            if (grid[row][col] == value)
                return false;
        }

        return true;
    }

    bool validInBlock(int row, int col, char value) const {

        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {

                if (grid[r][c] == value)
                    return false;
            }
        }

        return true;
    }

    bool isValidMove(int row, int col, char value) const {

        return validInRow(row, value) &&
               validInColumn(col, value) &&
               validInBlock(row, col, value);
    }

    // Check that the initial given clues do not violate Sudoku rules
    bool isValidInitialBoard() const {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                char val = grid[row][col];
                if (val != '_') {
                    // Check for invalid characters (must be '1'-'9')
                    if (val < '1' || val > '9') {
                        return false;
                    }
                    // Check for duplicate in the same row
                    for (int c = 0; c < 9; c++) {
                        if (c != col && grid[row][c] == val)
                            return false;
                    }
                    // Check for duplicate in the same column
                    for (int r = 0; r < 9; r++) {
                        if (r != row && grid[r][col] == val)
                            return false;
                    }
                    // Check for duplicate in the same 3x3 block
                    int startRow = (row / 3) * 3;
                    int startCol = (col / 3) * 3;
                    for (int r = startRow; r < startRow + 3; r++) {
                        for (int c = startCol; c < startCol + 3; c++) {
```

```
                            if ((r != row || c != col) && grid[r][c] == val)
                                return false;
                        }
                    }
                }
            }
        }
        return true;
    }

    void setValue(int row, int col, char value) {
        grid[row][col] = value;
    }

    vector<vector<char>> getGrid() const {
        return grid;
    }
};

class SudokuSolver {
private:
    vector<vector<vector<char>>> solutions;

public:

    void solve(Sudoku& puzzle) {

        int row;
        int col;

        if (!puzzle.findEmptyCell(row, col)) {

            solutions.push_back(puzzle.getGrid());
            return;
        }

        for (char digit = '1'; digit <= '9'; digit++) {

            if (puzzle.isValidMove(row, col, digit)) {

                puzzle.setValue(row, col, digit);

                solve(puzzle);

                // Backtracking step
                puzzle.setValue(row, col, '_');
            }
        }
    }

    void printSolutions() const {

        if (solutions.empty()) {

            cout << "\nNo solution found.\n";
            return;
        }

        for (size_t i = 0; i < solutions.size(); i++) {

            cout << "\nSolution " << i + 1 << ":\n\n";

            for (int row = 0; row < 9; row++) {

                for (int col = 0; col < 9; col++) {

                    cout << solutions[i][row][col] << " ";
                }

                cout << endl;
            }
        }
```

```
        cout << "\nSolutions found: "
             << solutions.size()
             << endl;
    }

    // Write all found solutions (or "No solution found.") to solutionN.txt
    void writeSolutionsToFile(const string& outputFilename) const {
        ofstream fout(outputFilename);

        if (!fout) {
            cout << "Error opening output file: " << outputFilename << endl;
            return;
        }

        // If no solutions were found, write the single required message line
        if (solutions.empty()) {
            fout << "No solution found." << endl;
            fout.close();
            return;
        }

        // Write each solution in the same 9x9 space-separated format as input files
        for (size_t i = 0; i < solutions.size(); i++) {
            for (int row = 0; row < 9; row++) {
                for (int col = 0; col < 9; col++) {
                    fout << solutions[i][row][col];
                    if (col < 8) {
                        fout << " ";
                    }
                }
                fout << endl;
            }
            // Add a blank line between multiple solutions for readability
            if (i + 1 < solutions.size()) {
                fout << endl;
            }
        }

        // Write the final summary line with the count of solutions found
        fout << "Solutions found: " << solutions.size() << endl;

        fout.close();
    }
};

class PuzzleManager {
private:
    vector<string> puzzleFiles;

    // Helper function to map "puzzleN.txt" to "solutionN.txt"
    string getSolutionFilename(const string& puzzleFilename) const {
        string prefix = "puzzle";
        if (puzzleFilename.find(prefix) == 0) {
            return "solution" + puzzleFilename.substr(prefix.length());
        }
        return "solution_" + puzzleFilename;
    }

public:

    PuzzleManager() {

        puzzleFiles.push_back("puzzle1.txt");
        puzzleFiles.push_back("puzzle2.txt");
        puzzleFiles.push_back("puzzle3.txt");
        puzzleFiles.push_back("puzzle4.txt");
        puzzleFiles.push_back("puzzle5.txt");
    }

    void processPuzzles() {

        for (string filename : puzzleFiles) {
```

```
            Sudoku puzzle;

            if (!puzzle.loadPuzzle(filename))
                continue;

            cout << "\n================================================";
            cout << "\nPuzzle File: " << filename;
            cout << "\n================================================\n";

            cout << "\nOriginal Puzzle:\n\n";
            puzzle.printBoard();

            SudokuSolver solver;
            // Only attempt to solve if the starting puzzle clues are valid
            if (puzzle.isValidInitialBoard()) {
                solver.solve(puzzle);
            }

            cout << "\nSolved Puzzle(s):\n";
            solver.printSolutions();

            // Write the solutions for this puzzle to its corresponding solutionN.txt file
            string solutionFilename = getSolutionFilename(filename);
            solver.writeSolutionsToFile(solutionFilename);

            cout << "\n";
        }
    }
};

int main() {

    PuzzleManager manager;

    manager.processPuzzles();

    return 0;
}
```

GenAI 2 — Claude Opus 5.5: complete, unedited reply

I fixed one bug that breaks the requirements and added the file output. I made no files, as you asked, so **I haven't compiled or run this**. Check it with `g++ -std=c++11 -Wall sudoku.cpp -o sudoku` on a cycle server before you submit.

Bug fixed:
- **Clashing starting numbers:** the original never checks the numbers already in the puzzle. If a file has the same digit twice in a row, column or 3×3 box (one way to make an unsolvable puzzle), it fills in the blanks around them and prints a wrong "solution". It now checks the starting numbers first and reports "No solution found." if they clash.
- **Bad input:** `loadPuzzle` now also rejects a file that is too short or contains anything other than `1`–`9` or `_`.

Changes for Assignment 5:
- **Output file:** `SudokuSolver` has a new `writeSolutions()` function. `processPuzzles()` works out the file name (`puzzleN.txt` → `solutionN.txt`) and calls it.
- **File format:** each solution is written like the input files: numbers separated by spaces, with no space at the end of a line. Solutions are separated by a blank line, and the file ends with `Solutions found: n`. If there are none, the file is just `No solution found.`

All original classes and lines are kept; new code is marked with `// Added:` comments. Fill in your name in the prologue.

```cpp
/*
 * Program: EECS 348 Assignment 5 - Recursive Sudoku Solver (file output)
 * Description: Reads puzzle1.txt - puzzle5.txt, prints each original puzzle,
 *              solves it with a recursive depth-first search with
 *              backtracking, prints every solution, and writes the solutions
 *              to solution1.txt - solution5.txt.
 * Original Author: (Assignment 4 author)
 * Revised by: [Your Name]
 * Revision Date: 10/29/2026
 * Revisions:
 *   - Added writing of all solutions to solutionN.txt (same format as the
 *     input files), or "No solution found." if there are none.
 *   - Added input validation in loadPuzzle (missing values / bad characters).
 *   - Added a check that the starting numbers do not already clash, so
 *     invalid puzzles correctly report "No solution found."
 */

#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

class Sudoku {
private:
    vector<vector<char>> grid;

public:
    Sudoku() {
        grid.resize(9, vector<char>(9));
    }

    bool loadPuzzle(const string& filename) {
        ifstream fin(filename);

        if (!fin) {
            cout << "Error opening file: " << filename << endl;
            return false;
        }

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                fin >> grid[row][col];

                // Added: make sure a value was actually read and that it is
                // either a digit 1-9 or an underscore (empty cell)
                if (!fin || !(grid[row][col] == '_' ||
```

```
                            (grid[row][col] >= '1' && grid[row][col] <= '9'))) {
                    cout << "Invalid puzzle format in file: " << filename << endl;
                    return false;
                }
            }
        }

        fin.close();
        return true;
    }

    void printBoard() const {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                cout << grid[row][col] << " ";
            }
            cout << endl;
        }
    }

    bool findEmptyCell(int& row, int& col) const {

        for (row = 0; row < 9; row++) {
            for (col = 0; col < 9; col++) {

                if (grid[row][col] == '_')
                    return true;
            }
        }

        return false;
    }

    bool validInRow(int row, char value) const {

        for (int col = 0; col < 9; col++) {
            if (grid[row][col] == value)
                return false;
        }

        return true;
    }

    bool validInColumn(int col, char value) const {

        for (int row = 0; row < 9; row++) {
            if (grid[row][col] == value)
                return false;
        }

        return true;
    }

    bool validInBlock(int row, int col, char value) const {

        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {

                if (grid[r][c] == value)
                    return false;
            }
        }

        return true;
    }

    bool isValidMove(int row, int col, char value) const {

        return validInRow(row, value) &&
               validInColumn(col, value) &&
```

```
                validInBlock(row, col, value);
    }

    // Added: checks that the starting (given) numbers do not already break
    // the Sudoku rules. Without this, a puzzle with duplicate givens could
    // be "solved" and report an invalid solution.
    bool hasValidGivens() {

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {

                char value = grid[row][col];

                if (value != '_') {
                    // Temporarily clear the cell so it is not compared with itself
                    grid[row][col] = '_';
                    bool ok = isValidMove(row, col, value);
                    grid[row][col] = value;   // restore the given value

                    if (!ok)
                        return false;
                }
            }
        }

        return true;
    }

    void setValue(int row, int col, char value) {
        grid[row][col] = value;
    }

    vector<vector<char>> getGrid() const {
        return grid;
    }
};

class SudokuSolver {
private:
    vector<vector<vector<char>>> solutions;

public:

    void solve(Sudoku& puzzle) {

        int row;
        int col;

        if (!puzzle.findEmptyCell(row, col)) {

            solutions.push_back(puzzle.getGrid());
            return;
        }

        for (char digit = '1'; digit <= '9'; digit++) {

            if (puzzle.isValidMove(row, col, digit)) {

                puzzle.setValue(row, col, digit);

                solve(puzzle);

                // Backtracking step
                puzzle.setValue(row, col, '_');
            }
        }
    }

    void printSolutions() const {

        if (solutions.empty()) {

            cout << "\nNo solution found.\n";
```

```
            return;
        }

        for (size_t i = 0; i < solutions.size(); i++) {

            cout << "\nSolution " << i + 1 << ":\n\n";

            for (int row = 0; row < 9; row++) {

                for (int col = 0; col < 9; col++) {

                    cout << solutions[i][row][col] << " ";
                }

                cout << endl;
            }
        }

        cout << "\nSolutions found: "
             << solutions.size()
             << endl;
    }

    // Added: writes all solutions to the given output file in the same
    // format as the input puzzle files (9 lines of 9 space-separated values),
    // followed by "Solutions found: n". If there are no solutions, writes
    // the single line "No solution found."
    void writeSolutions(const string& outFilename) const {

        ofstream fout(outFilename);

        // Added: report an error if the output file cannot be created
        if (!fout) {
            cout << "Error opening output file: " << outFilename << endl;
            return;
        }

        // Added: no solutions -> single line message
        if (solutions.empty()) {
            fout << "No solution found." << endl;
            fout.close();
            return;
        }

        // Added: write every solution grid, separated by a blank line
        for (size_t i = 0; i < solutions.size(); i++) {

            if (i > 0)
                fout << endl;

            for (int row = 0; row < 9; row++) {
                for (int col = 0; col < 9; col++) {

                    fout << solutions[i][row][col];

                    // Added: space between values, none at the end of a line
                    if (col < 8)
                        fout << " ";
                }
                fout << endl;
            }
        }

        // Added: final line with the number of solutions found
        fout << endl << "Solutions found: " << solutions.size() << endl;

        fout.close();
    }
};

class PuzzleManager {
private:
    vector<string> puzzleFiles;
```

```
public:

    PuzzleManager() {

        puzzleFiles.push_back("puzzle1.txt");
        puzzleFiles.push_back("puzzle2.txt");
        puzzleFiles.push_back("puzzle3.txt");
        puzzleFiles.push_back("puzzle4.txt");
        puzzleFiles.push_back("puzzle5.txt");
    }

    void processPuzzles() {

        for (string filename : puzzleFiles) {

            Sudoku puzzle;

            if (!puzzle.loadPuzzle(filename))
                continue;

            cout << "\n================================================";
            cout << "\nPuzzle File: " << filename;
            cout << "\n================================================\n";

            cout << "\nOriginal Puzzle:\n\n";
            puzzle.printBoard();

            SudokuSolver solver;

            // Added: only search for solutions if the starting numbers are
            // valid; otherwise the solver keeps zero solutions
            if (puzzle.hasValidGivens())
                solver.solve(puzzle);

            cout << "\nSolved Puzzle(s):\n";
            solver.printSolutions();

            // Added: build the output file name, e.g. puzzle1.txt -> solution1.txt
            // ("puzzle" is 6 characters, so substr(6) gives "N.txt")
            string solutionFile = "solution" + filename.substr(6);

            // Added: write the solutions (or "No solution found.") to the file
            solver.writeSolutions(solutionFile);
            cout << "\nSolutions written to: " << solutionFile << "\n";

            cout << "\n";
        }
    }
};

int main() {

    PuzzleManager manager;

    manager.processPuzzles();

    return 0;
}
```

If your instructor wants nothing between solutions, delete the `if (i > 0) fout << endl;` line and the `endl <<` before `"Solutions found: "` in `writeSolutions()`.

e. Analysis: Correctness, Execution Time, Space Complexity, Maintainability

How I tested. I compiled the original Assignment 4 code and both GenAI programs with `g++ -std=c++11` (g++ 15.2, Ubuntu Linux under WSL on my computer) and ran three test sets:

- **T1 – Assignment puzzles:** puzzle1.txt – puzzle5.txt. To check the solution files, I wrote a separate Python Sudoku solver that counts every solution and compares it with each solutionN.txt.
- **T2 – Bad input:** (1) a puzzle whose given numbers clash (two 7s in one row); (2) a file cut off after 5 rows; (3) a file with a `0` where an empty cell belongs; (4) a missing puzzle file; (5) a file with an extra 10th row.
- **T3 – Stress test:** a puzzle with rows 5–9 left empty, which has **144,720** solutions. I ran each program 3 times and report the median wall time and peak memory (`/usr/bin/time -v`). Console output went to a file.

1. Correctness

T1 – Assignment puzzles. The independent solver finds 1, 1, 1, 0 and 2 solutions for puzzles 1–5. Both GenAI programs found exactly these solutions and wrote matching solution1.txt – solution5.txt files (5 of 5 correct each). Both still print every Assignment 4 item (file name, original puzzle, each solution, the count) and both keep the recursive object-oriented design.

Puzzle	Correct # of solutions	Gemini 4 Argon solutionN.txt	Claude Opus 5.5 solutionN.txt
puzzle1.txt	1	Correct	Correct
puzzle2.txt	1	Correct	Correct
puzzle3.txt	1	Correct	Correct
puzzle4.txt	0	"No solution found."	"No solution found."
puzzle5.txt	2	Both solutions + "Solutions found: 2"	Both solutions + "Solutions found: 2"

File layout. In the puzzle files (and the instructor's sample solution file from Assignment 4), every value is followed by a space, and blank lines come before the count. Both GenAIs dropped the space after the last value of each row. Gemini puts a blank line only *between* solutions, so "Solutions found" sits directly under the last grid. Claude puts a blank line after every solution, which is closer to the sample.

T2 – Bad input. The original Assignment 4 code never checks its input, which produces wrong answers that both GenAIs partly fixed:

Test case	Original Assignment 4	Gemini 4 Argon	Claude Opus 5.5
Given numbers clash	Prints an invalid "solution" (its row 3 is 4 7 2 1 7 6 8 9 5 – two 7s)	"No solution found." (file written)	"No solution found." (file written)
File cut off after 5 rows	Unread cells stay '\0', which counts as "filled", so it prints 39,124 bogus "solutions" (24 MB of memory)	"No solution found." in the file, but no explanation. It is caught only by accident: '\0' fails the 1–9 test in isValidInitialBoard().	Prints "Invalid puzzle format in file", but writes no solution2.txt at all

'0' used for a blank	Treats 0 as a given and prints a "solution" whose first row is 5 0 9 8 2 4 1 7 6	"No solution found." (no explanation)	"Invalid puzzle format" message, no solution3.txt
Missing puzzle file	"Error opening file", no output	"Error opening file", no solution file	"Error opening file", no solution file
Extra 10th row	Silently uses the first 9 rows	Silently uses the first 9 rows	Silently uses the first 9 rows

Correctness summary: the two are tied on the real puzzles. On bad input, Claude finds every bad file on purpose and says why, but then skips writing its solution file. Gemini always writes a file but does not tell the user what was wrong, and it relies on an accident for short files. Neither rejects extra values.

2. Execution Time

Big-O. Neither GenAI changed the search, so all three programs share the same time complexity. The recursive depth-first search tries up to 9 digits in each of the m empty cells, so the worst case is $\mathbf{O(9^m)}$ search steps. Each step costs O(81) for `findEmptyCell()` (it rescans from the top-left corner) plus up to 9 × O(27) for the row, column and block checks. Both GenAIs add a one-time givens check of 81 × 27 comparisons, which is negligible. They also add file writing of O(81) characters per solution.

Hidden costs. All three copy the whole grid for every solution (`getGrid()` builds 10 new vectors each time) and print with `endl`, which flushes the stream after every line. The GenAIs add another `endl` per line in the file, so each solution costs about 20 flushes.

Measurement	Original A4 (screen only)	Gemini 4 Argon	Claude Opus 5.5
T1: all five assignment puzzles	0.05 s	< 0.2 s	< 0.2 s
T3: 144,720 solutions, median of 3 runs	7.39 s	8.63 s	8.41 s
T3 runs (min – max)	7.23 – 7.44 s	8.49 – 9.54 s	8.40 – 8.57 s

Both GenAI versions are about 14–17% slower than the original on T3 because they also write a 23.6 MB solution file, flushing each line. On the real puzzles, the difference cannot be noticed.

3. Space Complexity

All three keep every solution in `vector<vector<vector<char>>> solutions` until the search ends, so memory is **O(S × 81)** for S solutions. That is on top of O(81) for the board and O(m) ≤ 81 recursion frames. Neither GenAI changed this.

Measurement	Original A4	Gemini 4 Argon	Claude Opus 5.5
T1 peak memory	4.1 MB	4.0 MB	3.9 MB
T3 peak memory (median)	79.6 MB	79.5 MB	79.5 MB

About 75 MB of extra memory for 144,720 grids is roughly 550 bytes per stored solution (81 characters plus the overhead of 10 small vectors). The memory used grows with every solution found.

4. Maintainability

Item	Gemini 4 Argon	Claude Opus 5.5
Original lines changed / lines added	2 / 105	2 / 110

Comment lines	14	22
Givens check	New 35-line `isValidInitialBoard()` that rewrites the row, column and block rules a second time	21-line `hasValidGivens()` that reuses the existing `isValidMove()` (one copy of the rules)
Input checks	Not in `loadPuzzle()`; bad files are only caught later	Checked inside `loadPuzzle()`, right where the value is read
Solution file name	Helper `getSolutionFilename()` checks the "puzzle" prefix	Hard-coded `filename.substr(6)` (the "magic number" 6)
Change tracking	Short comments on new code	Every addition marked "// Added:", a revision list in the prologue, and a written summary of the changes
Prologue	Name, description, input, output; "[Your Name]" placeholder; no collaborators or sources	Same items plus a revision list; "[Your Name]" placeholder; no collaborators or sources
Const-correctness	`isValidInitialBoard()` is `const`	`hasValidGivens()` briefly edits the grid, so it is not `const`
Comments on the original code	None added (the original had 1 comment)	None added

f. Selection and Justification

I chose the Claude Opus 5.5 program as my base. The measurements gave a tie on the core requirements, so the choice depended on robustness and maintainability:

- **Tie on core results:** both wrote all 5 solution files correctly (T1). They were within 3% on time (8.41 s vs 8.63 s) and identical on memory (79.5 MB) in T3.
- **Better input handling:** Claude checks the input inside `loadPuzzle()` and tells the user what is wrong for the clash, short-file and bad-character cases (3 of 3). Gemini reports none of them and only catches the short file by accident.
- **Less duplicate code:** Claude's givens check reuses `isValidMove()` (21 lines vs 35). If a rule ever changes, it only has to change in one place.
- **Closer output layout:** Claude's blank line before "Solutions found" matches the instructor's sample solution file. Gemini's does not.
- **Easier to review with GitHub Blame:** every addition is labelled, with 22 comment lines vs 14.

The weaknesses I accepted and fixed in (g): unreadable puzzles get no solution file; the hard-coded `substr(6)`; extra values are accepted; every solution is stored; and output is flushed on every line.

g. Improvements I Made to the Claude Opus 5.5 Program

All changes are in Assignment5.cpp. Each block is tagged **[A4]** (original author), **[GenAI]** (Claude Opus 5.5) or **[Rev]** (my revision, written with Claude Code). I edited the received code instead of replacing it: 179 of its 218 lines (123 of 153 non-blank lines) are still in the file unchanged.

1. Correctness

Change	Effect (measured)
The solution file is created *before* the puzzle is read, and `printSummary()` always writes the final line	T2: every puzzle now gets solutionN.txt (**5 of 5** files vs 2 of 5 for the Claude base), each bad file with "No solution found." and a reason on screen
`loadPuzzle()` now also rejects a file with more than 81 values	10-row file: the base printed "Solutions found: 1"; the final prints "Invalid puzzle format" and writes "No solution found."
Solution rows are written by the same `printBoard()` that prints the puzzle	Each solution row now has exactly the puzzle-file layout (`5 3 9 8 2 4 1 7 6 `, each value followed by a space), with a blank line after each solution, like the instructor's sample
`substr(6)` replaced by a "puzzle"-prefix check (idea adapted from Gemini's helper)	A file name that does not start with "puzzle" can never produce a wrong or overwritten file name
Clashing givens now print "The given numbers break the Sudoku rules."	The user knows why there is no solution
Regression checks	T1: all five files still match the independent solver. T3: the final and the base produce the same 144,720 solutions (compared with `diff`, ignoring the trailing space).

2. Execution Time

- **`endl` → `'\n'`** in `printBoard()` and in the file output. Per solution this removes about 20 flushes (10 for the screen, 10 for the file); the stream is flushed once per puzzle by the summary line instead.
- **No grid copy per solution:** solutions are printed and written straight from the board, and `getGrid()` was removed. This saves 10 vector allocations per solution, plus the copying of every stored grid each time the solutions vector grew (about 17 times for 144,720 grids).
- **`for (const string& filename : puzzleFiles)`** no longer copies each file name.

Measurement	Claude base	Final Assignment5	Change
T3 median time (3 runs)	8.41 s	7.28 s	−13%
T3 runs (min – max)	8.40 – 8.57 s	6.36 – 7.79 s	
T1 (five assignment puzzles)	< 0.2 s	< 0.01 s	

The final version even beats the original Assignment 4 (7.39 s), although it does more work: it also writes a 24.9 MB solution file. The Big-O time is unchanged at $O(9^m)$, and that was a deliberate choice. Most of the remaining time is the search itself (sending the screen output to /dev/null did not make it faster). Making each rule check O(1) with bitmasks would mean replacing most of the original author's rule-check code, which goes

against "modify, don't replace" and against keeping the code easy to read. The real puzzles already finish in under 0.01 s.

3. Space Complexity

The solver no longer keeps a list of solutions. It keeps only a counter and a reference to the open solution file, so memory drops from **O(S × 81)** to **O(81)** for the board plus O(m) ≤ 81 recursion frames, whatever the number of solutions.

Measurement	Claude base	Final Assignment5	Change
T3 peak memory (median)	79.5 MB	4.15 MB	19× less
T1 peak memory	3.9 MB	4.1 MB	same

About 4 MB is the fixed size of a small C++ program, so memory no longer grows with the number of solutions.

4. Maintainability

- **Complete prologue:** program name, description, inputs, outputs, original author and creation date, "Revised by: Justin Whitaker" and the revision date, collaborators, every other source (both GenAIs and Claude Code), a block-tag legend, a revision list, and build/run commands.
- **Authorship on every block:** each block is tagged [A4], [GenAI] or [Rev]. I rewrote the GenAI's comments in my own words. Detailed block comments explain each original line *without editing those lines*, so GitHub Blame still credits the original author.
- **One formatting routine:** `printBoard(ostream& out = cout)` prints to the screen and writes to the file, so the file layout cannot drift from the puzzle layout. The base had a second, separate formatting loop for the file.
- **Clearer names, no dead code:** `printSolutions()` became `printSummary()` (what it now does), the unused `getGrid()` was removed, and the magic number 6 is gone.
- **Clean build:** no warnings with `g++ -std=c++11 -Wall -Wextra -pedantic`. C++11 keeps the code compatible with older compilers on the cycle servers.
- **Portable executable:** Assignment5.exe is statically linked (`g++ -std=c++11 -static`). A normal build on my machine needs GLIBC 2.34 or newer, which an older cycle server may not have.

Final Results (Assignment5.exe)

Puzzle	Terminal output	solutionN.txt (verified against the independent solver)
puzzle1.txt	1 solution	solution1.txt: 1 grid + "Solutions found: 1"
puzzle2.txt	1 solution	solution2.txt: 1 grid + "Solutions found: 1"
puzzle3.txt	1 solution	solution3.txt: 1 grid + "Solutions found: 1"
puzzle4.txt	"No solution found."	solution4.txt: "No solution found."
puzzle5.txt	2 solutions	solution5.txt: 2 grids + "Solutions found: 2"